Cobra Architecture Garden - Complete Edition

Evidence-backed design patterns from spf13/cobra

Analysis date: 2026-08-16; source commit: adbc8813901bba65827259daa8e22ff94ec1f30e

Contents

This compiled edition contains the project study, back-of-the-book design-pattern index, index rationale, and all detailed design notes. The source Markdown files retain their full Garden YAML frontmatter; this PDF-oriented compilation omits repeated YAML blocks and instead records each source file path before its rendered note.

Source repository: `https://github.com/spf13/cobra`
Pinned commit: `adbc8813901bba65827259daa8e22ff94ec1f30e`
Commit date: `2026-07-11`

Source file: *README.md*

Architecture Garden - Cobra

`spf13/cobra` is a command framework whose deepest reusable idea is not any one flag helper or shell script. Its architecture centers on a semantic command tree that is interpreted by routing, help, completion, validation, and documentation machinery, with hierarchy also carrying scoped flags, hooks, context, and I/O dependencies.

This study analyzes the repository at commit `adbc8813901b`. It follows the PARC Architecture Garden discipline: name invariants rather than mechanisms, prefer runtime code and tests over comments, record negative space, pin the evidence snapshot, and keep maturity claims conservative.

> [!summary] - Cobra's command tree is a semantic spine used by several projections, reducing drift between runtime and tooling. - Dispatch is root-owned while the selected leaf owns its validation and action lifecycle. - Hierarchy is used as a scope mechanism for persistent flags, hooks, context, and I/O. - Framework defaults are synthesized late so application-defined help/completion behavior can win. - Completion is an in-band protocol: shells ask the binary for semantic candidates and receive explicit directives. - Flag-group constraints are encoded once and projected into both runtime rejection and completion guidance. - Positional validation uses first-class functions and combinators. - A 2026 completion defect exposes an important Go ownership law: borrowed slices must not be mutated through shared backing arrays.

Snapshot identity and evidence

Field	Value
Repository	`spf13/cobra`
Remote	https://github.com/spf13/cobra
Branch	`main`
Commit	`adbc8813901bba65827259daa8e22ff94ec1f30e`
Commit date	2026-07-11
Commit subject	`fix: resolve macOS test link failure and update lint rules (#2429)`
Analysis access	GitHub connector snapshot; no local worktree assumptions
Primary implementation	`command.go`, `completions.go`, `args.go`, `flag_groups.go`, `cobra.go`, `doc/`
Primary tests	`command_test.go`, `completions_test.go`, `flag_groups_test.go`

The evidence order used here matches the Garden playbook: runtime code/public interfaces first, then tests, then repository history for failure-derived claims. The April 2026 completion mutation fix is included because it carries a concrete pre-fix failure, minimal repair, and regression test.

Architecture in one diagram

```
flowchart TD
    ARGV[argv + context + IO] --> ROOT[Root ExecuteC]
    ROOT --> FIND[Find / Traverse]
    FIND --> LEAF[Selected Command]
    LEAF --> PARSE[Parse effective flags]
    PARSE --> VALIDATE[Args + required flags + flag groups]
    VALIDATE --> HOOKS[Persistent/local hooks]
    HOOKS --> RUN[Run / RunE]

    TREE[Command tree] --> ROOT
    TREE --> HELP[Help + usage projection]
    TREE --> COMP[Completion projection]
```

```
    TREE --> DOCS[Markdown/man/YAML/reST docs]

    META[Flag/group metadata] --> VALIDATE
    META --> COMP
    IO[SetArgs / SetContext / SetIn / SetOut / SetErr] --> ARGV
```

Candidate common vocabulary

Proposed term	Cobra mechanism	Invariant
Semantic command graph	`Command`, `commands`, `parent`	One rooted model names executable CLI capabilities and their ancestry.
Dispatch root	`Root().ExecuteC()`	Global route selection and execution setup have one owner.
Leaf action	`cmd.execute` + `Run/RunE`	The selected operation validates and runs at the leaf.
Ancestry middleware	persistent pre/post hooks	Cross-cutting behavior follows declared command scope and order.
Cascading declaration	persistent flag	An outer declaration is visible to descendants.
Local shadow	local flag with inherited name	An inner scope can replace an inherited declaration in its effective view.
Synthetic affordance	default help/version/completion nodes	Framework behavior is materialized lazily through ordinary model objects.
Completion protocol	`__complete` + `ShellCompDirective`	External shell adapters query the binary for semantic completion.
Constraint projection	flag-group annotations	One relationship model drives both validation and guidance.
Borrowed input	argv slice	Caller-owned storage is not framework-owned mutable scratch space.

Pattern laws at a glance

- **Command Tree as Semantic Spine:** Declare a user-visible command capability once in a semantic command graph, then derive dispatch, help, completion, and generated documentation from that same graph instead of maintaining parallel inventories.
- **Root-Owned Dispatch, Leaf-Owned Action:** Route and orchestration belong to one root command; after selection, the leaf command owns argument validation and action execution while inherited context/policy flows to it.
- **Ancestor Hook Sandwich for Cross-Cutting Behavior:** Wrap a selected leaf action with explicitly ordered pre/post hooks whose scope follows command ancestry; make compatibility semantics for which ancestors run explicit rather than accidental.
- **Hierarchical Flags with Local Shadowing:** Configuration declared persistent on an ancestor flows to descendants, configuration declared local stays local, and a descendant-local name shadows the inherited name when views are assembled.
- **Late-Bound Synthetic Affordances Preserve User Override:** Framework-provided affordances should be injected as late as practical, only when absent, and preferably as ordinary model objects so user-defined behavior wins and existing traversal logic can be reused.
- **Completion as an In-Band Protocol:** Let the shell delegate semantic completion back to the executable through a stable hidden command protocol; return candidates as data plus an explicit directive so shell adapters do not need to duplicate application semantics.
- **Constraint Metadata Drives Both Rejection and Guidance:** Encode declarative relationships once, then interpret them both as runtime validity checks and as interactive guidance so the user interface does not suggest states that the validator will later reject.

- **First-Class Validators and Combinators:** Represent argument admissibility as a first-class function and compose small validators to express conjunctions, keeping route selection separate from command-specific argument policy.
- **Injectable Process Boundary for Deterministic Execution:** Treat process inputs and outputs as overridable execution dependencies - argv, context, stdin, stdout, and stderr - and make child operations inherit those seams unless they explicitly override them.
- **Borrowed Input Slices Stay Read-Only:** When a framework receives a caller-owned slice without ownership transfer, treat its backing storage as immutable; copy before any append or mutation whose capacity behavior could write into the caller's array.

1. Command Tree as Semantic Spine

Law. Declare a user-visible command capability once in a semantic command graph, then derive dispatch, help, completion, and generated documentation from that same graph instead of maintaining parallel inventories.

Cobra makes `Command` the shared semantic object for both execution and user-facing projections. The reusable pattern is not merely a tree of handlers; it is one model interpreted several ways.

Primary evidence: `command.go`, `completions.go`, `doc/md_docs.go`.

Maturity: **Candidate ecosystem pattern**.

Detailed note: 01 - Command Tree as Semantic Spine.

2. Root-Owned Dispatch, Leaf-Owned Action

Law. Route and orchestration belong to one root command; after selection, the leaf command owns argument validation and action execution while inherited context/policy flows to it.

Cobra re-anchors execution at the root even when `ExecuteC` is called on a child. The root resolves the path and the selected leaf executes its local lifecycle.

Primary evidence: `command.go`, `command_test.go`.

Maturity: **Established locally**.

Detailed note: 02 - Root-Owned Dispatch, Leaf-Owned Action.

3. Ancestor Hook Sandwich for Cross-Cutting Behavior

Law. Wrap a selected leaf action with explicitly ordered pre/post hooks whose scope follows command ancestry; make compatibility semantics for which ancestors run explicit rather than accidental.

Cobra provides local and persistent run hooks. With traversal enabled, persistent pre-hooks run root-to-leaf and persistent post-hooks leaf-to-root around the leaf action.

Primary evidence: `command.go`, `cobra.go`, `command_test.go`.

Maturity: **Established locally**.

Detailed note: 03 - Ancestor Hook Sandwich for Cross-Cutting Behavior.

4. Hierarchical Flags with Local Shadowing

Law. Configuration declared persistent on an ancestor flows to descendants, configuration declared local stays local, and a descendant-local name shadows the inherited name when views are assembled.

Cobra turns the command ancestry into a configuration scope tree. Persistent flags are merged downward, while local flags can intentionally shadow inherited flags.

Primary evidence: `command.go`, `command_test.go`.

Maturity: **Established locally**.

Detailed note: 04 - Hierarchical Flags with Local Shadowing.

5. Late-Bound Synthetic Affordances Preserve User Override

Law. Framework-provided affordances should be injected as late as practical, only when absent, and preferably as ordinary model objects so user-defined behavior wins and existing traversal logic can be reused.

Cobra lazily creates help/version/completion affordances and checks for user overrides first. The defaults enter the same command/flag model rather than bypassing it.

Primary evidence: `command.go`, `completions.go`.

Maturity: **Candidate ecosystem pattern**.

Detailed note: 05 - Late-Bound Synthetic Affordances Preserve User Override.

6. Completion as an In-Band Protocol

Law. Let the shell delegate semantic completion back to the executable through a stable hidden command protocol; return candidates as data plus an explicit directive so shell adapters do not need to duplicate application semantics.

Cobra shell scripts call hidden commands inside the program. The program resolves the real command context and returns completion candidates plus a bitmapped directive over stdout.

Primary evidence: `completions.go`, `completions_test.go`, `command.go`.

Maturity: **Established locally**.

Detailed note: 06 - Completion as an In-Band Protocol.

7. Constraint Metadata Drives Both Rejection and Guidance

Law. Encode declarative relationships once, then interpret them both as runtime validity checks and as interactive guidance so the user interface does not suggest states that the validator will later reject.

Cobra stores flag-group relationships as annotations, validates those annotations before running, and also interprets them during completion to require or hide related flags.

Primary evidence: `flag_groups.go`, `flag_groups_test.go`, `completions.go`.

Maturity: **Candidate ecosystem pattern**.

Detailed note: 07 - Constraint Metadata Drives Both Rejection and Guidance.

8. First-Class Validators and Combinators

Law. Represent argument admissibility as a first-class function and compose small validators to express conjunctions, keeping route selection separate from command-specific argument policy.

Cobra defines `PositionalArgs` as a function type and supplies reusable validators plus `MatchAll`, which short-circuits through a sequence of validators.

Primary evidence: `args.go`, `command.go`.

Maturity: **Established locally**.

Detailed note: 08 - First-Class Validators and Combinators.

9. Injectable Process Boundary for Deterministic Execution

Law. Treat process inputs and outputs as overridable execution dependencies - argv, context, stdin, stdout, and stderr - and make child operations inherit those seams unless they explicitly override them.

Cobra defaults to `os.Args`, stdin/stdout/stderr, and a background context, but exposes setters and ancestor fallback so the same command graph can execute in tests or embedded hosts.

Primary evidence: `command.go`, `command_test.go`.

Maturity: **Candidate ecosystem pattern**.

Detailed note: 09 - Injectable Process Boundary for Deterministic Execution.

10. Borrowed Input Slices Stay Read-Only

Law. When a framework receives a caller-owned slice without ownership transfer, treat its backing storage as immutable; copy before any append or mutation whose capacity behavior could write into the caller's array.

A Cobra completion bug showed that slicing `os.Args` and later appending a sentinel could overwrite the shared backing array. The fix copies into new storage before speculative append and adds a regression test.

Primary evidence: `completions.go`, `completions_test.go`.

Maturity: **Candidate ecosystem pattern**.

Detailed note: 10 - Borrowed Input Slices Stay Read-Only.

Maturity assessment

Pattern	Maturity	Evidence or limitation
Command Tree as Semantic Spine	Candidate ecosystem pattern	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
Root-Owned Dispatch, Leaf-Owned Action	Established locally	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
Ancestor Hook Sandwich for Cross-Cutting Behavior	Established locally	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
Hierarchical Flags with Local Shadowing	Established locally	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
Late-Bound Synthetic Affordances Preserve User Override	Candidate ecosystem pattern	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
Completion as an In-Band Protocol	Established locally	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
Constraint Metadata Drives Both Rejection and Guidance	Candidate ecosystem pattern	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
First-Class Validators and Combinators	Established locally	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
Injectable Process Boundary for Deterministic Execution	Candidate ecosystem pattern	Direct runtime implementation plus tests/documentation; broader reuse claim kept conservative.
Borrowed Input Slices Stay Read-Only	Candidate ecosystem pattern	Failure-derived from a fixed mutation bug and regression test.

Failure modes and design tensions

Completion must not mutate caller-owned argv

Commit `746ef0715872` fixed a case where a sub-slice derived from `os.Args` shared spare backing-array capacity and a later `append(..., "--")` could overwrite a caller-visible argument. The current implementation allocates a new slice before speculative append, and the regression test asserts that `os.Args` remains unchanged.

Lifecycle traversal is compatibility-sensitive

`EnableTraverseRunHooks` is a package-level switch and defaults to false. Full root-to-leaf persistent pre-hook traversal and leaf-to-root persistent post-hook traversal are therefore an opt-in semantic mode; the legacy mode executes only the first persistent hook found in each direction. Consumers should not describe "all ancestors always run" as an unconditional Cobra law.

Completion guidance is not authority

`ValidArgs`, completion functions, and flag-group completion behavior improve guidance, but runtime admissibility is still decided by argument and flag validation. Suggested does not mean authorized; unsuggested does not necessarily mean invalid.

The semantic tree is mutable

Default help/completion behavior is synthesized at execution/help time. The command graph is authoritative, but not immutable. Systems that require concurrent immutable introspection should adapt the pattern by compiling a frozen execution snapshot.

Related studies and method

- Index of Design Patterns - back-of-the-book retrieval index and notation table.
- Index of Design Patterns - Rationale - why each access term and maturity label was selected.
- PARC playbook: Creating GitHub Issues and Software Design Garden Entries - used for evidence order, invariant naming, negative space, and maturity discipline.
- PARC playbook: Creating an Index for a Software Architecture Garden Entry - used for index/glossary/notation separation and alternate access paths.
- PARC Architecture Garden - sessionstream - used as a concrete project/design-note style reference.

Source inventory

- `README.md` - project concepts and advertised capability surface.
- `command.go` - command model, routing, execution lifecycle, help/usage, flags, I/O/context seams.
- `cobra.go` - package-level compatibility switches and global initializers/finalizers.
- `args.go` - positional validator functions and `MatchAll`.
- `completions.go` - hidden completion protocol, directives, completion resolution, default completion command.
- `flag_groups.go` - declarative flag relationships, validation, completion projection.
- `doc/md_docs.go` - documentation traversal over the same command graph.
- `command_test.go` - execution, context, aliases, hook and flag-scope behavior.
- `completions_test.go` - completion protocol and semantic completion behavior.
- `flag_groups_test.go` - flag relationship validation across local/persistent/inherited scopes.
- Pinned commit.
- Borrowed-slice regression commit.

Source file: *Index of Design Patterns.md*

Cobra - Index of Design Patterns

This is a retrieval index, glossary, and notation table for the Cobra Architecture Garden study. Canonical entries are filed by the invariant a reader is likely to remember; alternate phrasings route through explicit `See` entries.

How to read this index

- Each canonical entry gives a one-sentence definition, maturity, a locator into the project study, and a link to the detailed design note.
- `See` means the alternate phrase should route to the canonical concept rather than maintain separate locators.
- `see also` links related-but-distinct patterns that should not be flattened into one.
- The notation table at the end is for versioned/symbolic handles and compatibility switches, not alphabetic concepts.

Canonical entries

Command Tree as Semantic Spine

Cobra makes `Command` the shared semantic object for both execution and user-facing projections. The reusable pattern is not merely a tree of handlers; it is one model interpreted several ways. [**Candidate ecosystem pattern**] §1 · design note. *see also* Late-Bound Synthetic Affordances Preserve User Override, Completion as an In-Band Protocol, Constraint Metadata Drives Both Rejection and Guidance.

Root-Owned Dispatch, Leaf-Owned Action

Cobra re-anchors execution at the root even when `ExecuteC` is called on a child. The root resolves the path and the selected leaf executes its local lifecycle. [**Established locally**] §2 · design note. *see also* Ancestor Hook Sandwich for Cross-Cutting Behavior, Injectable Process Boundary for Deterministic Execution.

Ancestor Hook Sandwich for Cross-Cutting Behavior

Cobra provides local and persistent run hooks. With traversal enabled, persistent pre-hooks run root-to-leaf and persistent post-hooks leaf-to-root around the leaf action. [**Established locally**] §3 · design note. *see also* Root-Owned Dispatch, Leaf-Owned Action, Injectable Process Boundary for Deterministic Execution.

Hierarchical Flags with Local Shadowing

Cobra turns the command ancestry into a configuration scope tree. Persistent flags are merged downward, while local flags can intentionally shadow inherited flags. [**Established locally**] §4 · design note. *see also* Constraint Metadata Drives Both Rejection and Guidance, Injectable Process Boundary for Deterministic Execution.

Late-Bound Synthetic Affordances Preserve User Override

Cobra lazily creates help/version/completion affordances and checks for user overrides first. The defaults enter the same command/flag model rather than bypassing it. [**Candidate ecosystem pattern**] §5 · design note. *see also* Command Tree as Semantic Spine, Completion as an In-Band Protocol.

Completion as an In-Band Protocol

Cobra shell scripts call hidden commands inside the program. The program resolves the real command context and returns completion candidates plus a bitmapped directive over stdout. [**Established locally**] §6 · design note. *see also* Command Tree as Semantic Spine, Late-Bound Synthetic Affordances Preserve User Override, Constraint Metadata Drives Both Rejection and Guidance, Borrowed Input Slices Stay Read-Only.

Constraint Metadata Drives Both Rejection and Guidance

Cobra stores flag-group relationships as annotations, validates those annotations before running, and also interprets them during completion to require or hide related flags. [**Candidate ecosystem pattern**] §7 · design note. *see also* Hierarchical Flags with Local Shadowing, Completion as an In-Band Protocol, First-Class Validators and Combinators.

First-Class Validators and Combinators

Cobra defines `PositionalArgs` as a function type and supplies reusable validators plus `MatchAll`, which short-circuits through a sequence of validators. [**Established locally**] §8 · design note. *see also* Constraint Metadata Drives Both Rejection and Guidance.

Injectable Process Boundary for Deterministic Execution

Cobra defaults to `os.Args`, stdin/stdout/stderr, and a background context, but exposes setters and ancestor fallback so the same command graph can execute in tests or embedded hosts. [**Candidate ecosystem pattern**] §9 · design note. *see also* Root-Owned Dispatch, Leaf-Owned Action, Hierarchical Flags with Local Shadowing, Borrowed Input Slices Stay Read-Only.

Borrowed Input Slices Stay Read-Only

A Cobra completion bug showed that slicing `os.Args` and later appending a sentinel could overwrite the shared backing array. The fix copies into new storage before speculative append and adds a regression test. [**Candidate ecosystem pattern**] §10 · design note. *see also* Completion as an In-Band Protocol, Injectable Process Boundary for Deterministic Execution.

Alternate access paths

Single source of truth for CLI metadata

See Command Tree as Semantic Spine. Use this access path when the remembered idea is preventing drift across help, completion, docs, and routing.

Command graph as AST

See Command Tree as Semantic Spine. The graph is interpreted by multiple projections.

Root orchestration

See Root-Owned Dispatch, Leaf-Owned Action. One root resolves; one leaf acts.

Middleware inheritance

See Ancestor Hook Sandwich for Cross-Cutting Behavior. Persistent hooks are ancestry-scoped middleware.

Global flags

See Hierarchical Flags with Local Shadowing. Cobra calls inherited persistent flags “Global Flags” in help output.

Flag shadowing

See Hierarchical Flags with Local Shadowing. A child-local declaration can hide an inherited name.

Lazy help command

See Late-Bound Synthetic Affordances Preserve User Override. Help/version/completion defaults are created only when needed and absent.

Synthetic commands

See Late-Bound Synthetic Affordances Preserve User Override. Framework behaviors are represented as ordinary commands.

Shell completion RPC

See Completion as an In-Band Protocol. The hidden command behaves like a tiny argv/stdout request protocol.

Completion directives

See Completion as an In-Band Protocol. Directive bits tell shell adapters how to treat returned candidates.

Mutually exclusive flags

See Constraint Metadata Drives Both Rejection and Guidance. Mutual exclusion is one flag-group relationship projected into validation and completion.

Required-together flags

See Constraint Metadata Drives Both Rejection and Guidance. The same group metadata drives rejection and next-token guidance.

Argument validation combinators

See First-Class Validators and Combinators. Use this for `MatchAll` and the `PositionalArgs` function family.

Test seams

See Injectable Process Boundary for Deterministic Execution. Args, context, and streams are overridable execution dependencies.

Dependency injection

See Injectable Process Boundary for Deterministic Execution. Process globals become fallbacks rather than mandatory dependencies.

os.Args mutation

See Borrowed Input Slices Stay Read-Only. Failure-derived access path for the 2026 completion bug.

Copy before append

See Borrowed Input Slices Stay Read-Only. A new backing array is required before speculative append to borrowed input.

Identity strings, handles, and compatibility switches

Handle	Kind	Meaning	Where
`Command`	semantic node	CLI capability plus metadata, hooks, flags, parent/children, context and I/O seams	§1, §2
`__complete`	hidden protocol command	Request completion candidates with descriptions	§6
`__completeNoDesc`	hidden protocol alias	Request completion candidates without descriptions	§6
`ShellCompDirective`	bitmap	Tells a shell whether to add spaces, fall back to files, filter dirs/extensions, keep order, or treat result as error	§6

Handle	Kind	Meaning	Where
`PositionalArgs`	function type	`(cmd,args) -> error` positional argument policy	§8
`CompletionFunc`	function type	Dynamic completion provider returning candidates plus directive	§6
`PersistentFlags()`	scope declaration	Flags that cascade from a command to descendants	§4
`InheritedFlags()`	effective view	Parent-persistent flags visible at a child after local shadowing	§4
`TraverseChildren`	command option	Parses flags on parents while traversing descendants	§2, §4, §6
`EnableTraverseRunHooks`	package compatibility switch	Enables all-ancestor persistent hook traversal	§3
`SilenceErrors` / `SilenceUsage`	presentation policy	Controls automatic error/usage printing around returned execution errors	§2

Failure modes and open obligations

Completion mutates caller arguments

See Borrowed Input Slices Stay Read-Only. The April 2026 fix turned a concrete `os.Args` corruption defect into an explicit ownership invariant.

All persistent hooks run

See Ancestor Hook Sandwich for Cross-Cutting Behavior. This is true only when `EnableTraverseRunHooks` is enabled; legacy default behavior runs only the first persistent hook found.

Completion equals validation

See Constraint Metadata Drives Both Rejection and Guidance, First-Class Validators and Combinators. Completion guidance and runtime authority are separate interpreters; `ValidArgs` alone does not enforce runtime validity.

Source file: *Index of Design Patterns - Rationale.md*

Cobra - Index of Design Patterns - Rationale

This note records why the index terms were selected, why each belongs, and how maturity was assigned. It is editorial evidence, not a second architecture study.

Principles of selection

1. Index protected invariants and operational consequences, not every exported identifier.
2. Prefer concepts that answer questions a maintainer will actually ask: "why do help and completion stay in sync?", "who owns dispatch?", "how do global flags scope?", "why can completion not mutate argv?".
3. Keep authority distinctions visible: completion is guidance, validation is authority; dispatch is not authorization; post hooks are not rollback; generated docs are not runtime proof.
4. Include failure-derived knowledge. The `os.Args` mutation bug belongs because it changed an ownership invariant and now has a regression test.
5. Use conservative maturity. Strong behavior inside one repository can be established locally; reuse beyond the repository remains a candidate until independently compared.

What was deliberately excluded

- Levenshtein suggestions were not promoted to a design entry: useful behavior, but the current study found no broader invariant beyond conventional typo assistance.
- Command grouping (`GroupID`) was kept as presentation metadata rather than a standalone pattern because it does not alter routing semantics.
- Template customization was not split into a separate entry; it is an extension surface subordinate to the broader semantic-tree and inherited-configuration patterns.
- Deprecated compatibility helpers were not indexed individually except where they reveal a stronger design transition (for example `ExactValidArgs` reducing to validator composition).
- POSIX flag parsing itself belongs primarily to `pflag`; this study focuses on Cobra's orchestration and reuse of those flag objects.

Command Tree as Semantic Spine

> Index entry: Command Tree as Semantic Spine.

Chosen because. Chosen because the same `Command` graph is consumed by execution, help, completion, and generated Markdown, making the anti-drift invariant more fundamental than any one mechanism.

Belongs because. Without it, the garden would list help, completion, and docs as unrelated features and miss the architectural reason they remain synchronized.

Maturity rationale. Classified as **Candidate ecosystem pattern**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Root-Owned Dispatch, Leaf-Owned Action

> Index entry: Root-Owned Dispatch, Leaf-Owned Action.

Chosen because. Chosen because `ExecuteC` explicitly re-anchors at the root and then delegates to one selected leaf, giving dispatch ownership a concrete runtime boundary.

Belongs because. Without it, root-vs-leaf ownership is easy to conflate with ordinary tree traversal, obscuring where global policy and context enter execution.

Maturity rationale. Classified as **Established locally**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Ancestor Hook Sandwich for Cross-Cutting Behavior

Index entry: Ancestor Hook Sandwich for Cross-Cutting Behavior.

Chosen because. Chosen because hook scope and order are encoded directly in the execution pipeline and tests, with an explicit compatibility switch that prevents oversimplifying the claim.

Belongs because. Without it, readers may import generic middleware assumptions that are false under Cobra's legacy hook mode or on error paths.

Maturity rationale. Classified as **Established locally**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Hierarchical Flags with Local Shadowing

Index entry: Hierarchical Flags with Local Shadowing.

Chosen because. Chosen because persistent/local/inherited flag views form a durable scope model and tests specifically cover local shadowing of inherited names.

Belongs because. Without it, "global flag" becomes a vague UX term and the crucial shadowing/origin rules disappear.

Maturity rationale. Classified as **Established locally**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Late-Bound Synthetic Affordances Preserve User Override

Index entry: Late-Bound Synthetic Affordances Preserve User Override.

Chosen because. Chosen because help, version, and completion independently demonstrate late synthesis plus user-override checks; the hidden completion command additionally shows why temporary framework nodes need bounded lifetime.

Belongs because. Without it, defaults look like boilerplate rather than an extensibility strategy that protects user-defined names and limits structural side effects.

Maturity rationale. Classified as **Candidate ecosystem pattern**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Completion as an In-Band Protocol

Index entry: Completion as an In-Band Protocol.

Chosen because. Chosen because the hidden completion command, line protocol, directive bitmap, and exact-output tests form a clear adapter protocol rather than a collection of shell-specific helpers.

Belongs because. Without it, shell completion appears to be generated script magic rather than a protocol boundary between shell mechanics and application semantics.

Maturity rationale. Classified as **Established locally**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Constraint Metadata Drives Both Rejection and Guidance

Index entry: Constraint Metadata Drives Both Rejection and Guidance.

Chosen because. Chosen because the same flag annotations feed both final validation and next-token completion behavior, demonstrating two projections over one constraint representation.

Belongs because. Without it, validation and completion look like separate subsystems and the shared-constraint anti-drift lesson is lost.

Maturity rationale. Classified as **Candidate ecosystem pattern**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

First-Class Validators and Combinators

Index entry: First-Class Validators and Combinators.

Chosen because. Chosen because `PositionalArgs` and `MatchAll` are a compact, explicit validation algebra, and deprecated specialized helpers are expressed through composition.

Belongs because. Without it, argument helpers read as convenience functions rather than a reusable boundary between routing and admissibility.

Maturity rationale. Classified as **Established locally**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Injectable Process Boundary for Deterministic Execution

Index entry: Injectable Process Boundary for Deterministic Execution.

Chosen because. Chosen because command tests use injected args, context, and I/O as the normal testing path, while parent fallback makes the seam hierarchical rather than merely ad hoc.

Belongs because. Without it, Cobra testing looks dependent on process globals even though its command object explicitly virtualizes them.

Maturity rationale. Classified as **Candidate ecosystem pattern**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Borrowed Input Slices Stay Read-Only

Index entry: Borrowed Input Slices Stay Read-Only.

Chosen because. Chosen because it is grounded in a historical defect, minimal repair, and regression test. Failure evidence makes the ownership law unusually concrete.

Belongs because. Without it, the study would flatter the architecture by listing only successful abstractions and would miss an ownership failure that materially changed the code.

Maturity rationale. Classified as **Candidate ecosystem pattern**. Direct Cobra behavior is treated as established where runtime code plus tests demonstrate it. Broader "ecosystem pattern" labels remain candidate because this deliverable studies one repository rather than performing multi-project validation.

Alternate-access rationale

The index includes redirects such as "single source of truth for CLI metadata", "root orchestration", "global flags", "shell completion RPC", "test seams", and "copy before append" because readers are likely to remember the operational problem rather than the Garden's canonical title. They route to canonical entries rather than duplicating locators.

Source file: *designs/01 - Command Tree as Semantic Spine.md*

Command Tree as Semantic Spine

Cobra makes `Command` the shared semantic object for both execution and user-facing projections. The reusable pattern is not merely a tree of handlers; it is one model interpreted several ways.

Pattern statement: Declare a user-visible command capability once in a semantic command graph, then derive dispatch, help, completion, and generated documentation from that same graph instead of maintaining parallel inventories.

1. Why this pattern exists

CLI frameworks accumulate representations quickly: a routing table, a help catalog, shell-completion rules, documentation metadata, aliases, flag definitions, and often a second structure for generated reference pages. When these structures are authored separately, they drift. A command can execute but disappear from help, completion can suggest syntax that runtime validation rejects, or generated docs can describe a different tree than the binary actually routes.

Cobra's central design choice is to make `Command` carry the durable semantics of a CLI node: its name/usage, aliases, descriptions, argument validator, completion metadata, run hooks, local and persistent flags, parent, and child commands. The same object graph is then traversed by multiple interpreters.

> [!summary] **Law:** Declare once; project many times. A command graph should be the source model for routing and user-facing tooling. Secondary views should traverse or interpret that graph rather than re-declare the command inventory.

2. The general pattern

Model the CLI as a rooted tree `T` whose nodes are semantic command descriptions. A projection is a function over that tree:

```
route      : (T, argv) -> selected command + remaining argv
help       : T -> human-readable usage
completion : (T, partial argv) -> candidates + shell directives
docs       : T -> reference-document tree
```

The important invariant is not that every projection emits the same text. It is that they consume the same identities, parent/child relations, visibility rules, descriptions, and flag scopes. This is the CLI form of a shared abstract syntax tree with multiple interpreters.

3. Cobra's concrete architecture

`Command` owns both structure (`commands`, `parent`) and behavioral/user-facing metadata. `AddCommand` establishes the parent relation. `ExecuteC` resolves through `Find` or `Traverse`, then invokes the selected command. Default help walks `Commands()` and consults `IsAvailableCommand`, flag visibility, groups, aliases, and descriptions. Completion resolves the same command path, then reads subcommands, flags, `ValidArgs`, and `ValidArgsFunction`. The `doc` package recursively traverses `Commands()` and emits Markdown pages from `CommandPath`, `Short`, `Long`, examples, and inherited/local flags.

Source evidence:

- `command.go` - `Command`, `AddCommand`, `Find`, `Traverse`, `ExecuteC`, default help/usage, flag scopes.
- `completions.go` - completion resolves the same tree and projects candidates from command/flag metadata.
- `doc/md_docs.go` - generated Markdown recursively traverses the command tree.
- `command_test.go` - child routing, aliases, context propagation, output seams, and other tree behavior are tested.

4. Behavioral contract

The reusable contract can be stated as four laws:

1. **Identity law.** A command's canonical identity and ancestry come from the graph used by dispatch.
2. **Projection law.** Help, completion, and docs obtain command identities and relationships from that graph.
3. **Visibility law.** Hidden/deprecated/help-topic status is interpreted by projections rather than copied into parallel catalogs.
4. **Extension law.** New subcommands enter the ecosystem by being attached to the graph; projections discover them through traversal.

These laws reduce synchronization work. They do not guarantee that every projection has identical filtering semantics, and they do not imply that metadata alone proves behavioral correctness.

5. Negative space

A command graph is not the application's domain model. Business state, authorization, durable workflows, or transaction ownership should not be pushed into `Command` merely because the graph is convenient. The tree describes the interaction surface and execution wiring.

Generated documentation is also not runtime evidence. It can prove that a node and its declared flags exist in the model; it cannot prove that a `RunE` handler performs the documented domain effect correctly.

Finally, Cobra mutates the graph at runtime to inject default help/completion affordances. Therefore "single source" means one authoritative semantic graph, not necessarily one immutable graph.

6. When to reuse

Reuse this pattern when a framework has one stable semantic inventory with several consumers: command routers, RPC schemas, job definitions, workflow nodes, UI action registries, or plugin manifests. It is especially valuable when documentation and interactive assistance must stay synchronized with executable behavior.

Avoid it when the projections need genuinely different domain objects. Forcing unrelated concepts into one mega-model can create a false unification that is harder to evolve than two explicitly related models.

7. Maturity

Candidate ecosystem pattern. Cobra demonstrates the invariant across routing, help, shell completion, and several documentation generators with extensive tests. The broader architectural generalization is strong, but this study has not compared multiple independent frameworks under the Garden's evidence rules.

Source map

- `command.go`
- `completions.go`
- `doc/md_docs.go`
- `README.md`
- `command_test.go`
- Pinned repository commit

Related designs

- Late-Bound Synthetic Affordances Preserve User Override
- Completion as an In-Band Protocol
- Constraint Metadata Drives Both Rejection and Guidance

Source file: *designs/02 - Root-Owned Dispatch, Leaf-Owned Action.md*

Root-Owned Dispatch, Leaf-Owned Action

Cobra re-anchors execution at the root even when `ExecuteC` is called on a child. The root resolves the path and the selected leaf executes its local lifecycle.

Pattern statement: Route and orchestration belong to one root command; after selection, the leaf command owns argument validation and action execution while inherited context/policy flows to it.

1. Why this pattern exists

Hierarchical command systems need a clear owner for dispatch. If every node can independently parse the whole process command line, initialize global facilities, and decide which descendant runs, behavior becomes sensitive to the object on which execution happened to be invoked.

Cobra avoids that ambiguity. `ExecuteC` checks whether the receiver has a parent and, if so, delegates to `Root().ExecuteC()`. The root performs late initialization, resolves the command path through `Find` or `Traverse`, propagates the root context to the selected command, and then calls that leaf's internal `execute` pipeline.

> [!summary] **Law:** One root owns route selection and global execution setup. A selected leaf owns its local validation and handler lifecycle. Calling execution on an interior node must not silently create a second routing universe.

2. The general pattern

Separate two responsibilities:

```
root orchestration
  - acquire argv / execution context
  - install framework defaults
  - resolve one target node
  - apply global error/usage policy
  - propagate root-scoped dependencies

leaf execution
  - parse applicable flags
  - validate positional arguments and flag constraints
  - run pre/action/post lifecycle
  - return an error to the orchestrator
```

The root is a control-plane owner; the leaf is the selected operation.

3. Cobra's concrete architecture

In command.go, `ExecuteC`:

- creates a background context when none was provided;
- re-anchors execution to `Root()` if invoked on a child;
- injects default help and completion affordances;
- resolves the target via `Find` or `Traverse`;
- records the name/alias used to call the target;
- propagates context from root to leaf when needed;
- invokes `cmd.execute(flags)`;
- applies root/leaf `SilenceErrors` and `SilenceUsage` policy around returned errors.

The internal `execute` function then parses target flags, handles help/version, checks runnability, validates args and flag groups, and runs the command hooks/handler.

Tests in command_test.go verify child selection through `ExecuteC`, execution context propagation down several levels, and consistent routing when aliases are used.

4. Invariants

R1 - Unique orchestration root. Dispatch always starts from the root graph.

R2 - One selected command. Resolution returns one concrete `Command` plus the argument slice to execute.

R3 - Root context reaches the leaf. A context attached to root execution is copied to the selected command when the leaf has none.

R4 - Presentation policy remains outside the leaf action. Error and usage emission are controlled by execution policy around the leaf's returned error, rather than requiring every handler to format CLI errors itself.

5. Negative space

This is not an authorization boundary. A leaf being selected by the root says nothing about whether the user is allowed to perform its domain action.

It also does not mean all cross-cutting behavior lives at root. Cobra's persistent hooks can be attached to ancestors, and persistent flags are inherited down the tree. The root owns *dispatch*, not every concern.

Finally, the root still contains mutable configuration and process-global compatibility switches elsewhere in the package. Root ownership creates a clear execution locus; it does not make the whole framework purely functional.

6. When to reuse

This pattern transfers well to plugin dispatchers, nested workflow engines, route trees, and build systems. It is useful whenever nested objects are independently addressable in the model but execution must have one authoritative entrance for global setup, telemetry, cancellation, and error policy.

7. Maturity

Established locally. The re-anchoring rule and root-to-leaf execution flow are direct runtime behavior with focused tests. Generalizing it beyond command frameworks remains a separate claim.

Source map

- `command.go`
- `command_test.go`
- Pinned repository commit

Related designs

- Ancestor Hook Sandwich for Cross-Cutting Behavior
- Injectable Process Boundary for Deterministic Execution

Source file: *designs/03 - Ancestor Hook Sandwich for Cross-Cutting Behavior.md*

Ancestor Hook Sandwich for Cross-Cutting Behavior

Cobra provides local and persistent run hooks. With traversal enabled, persistent pre-hooks run root-to-leaf and persistent post-hooks leaf-to-root around the leaf action.

Pattern statement: Wrap a selected leaf action with explicitly ordered pre/post hooks whose scope follows command ancestry; make compatibility semantics for which ancestors run explicit rather than accidental.

1. Why this pattern exists

Nested commands often share setup and teardown concerns: configuration loading, tracing, authentication context preparation, resource acquisition, metrics, or cleanup. Copying those concerns into every leaf creates drift; hiding them in global initializers makes scope unclear.

Cobra models cross-cutting execution as hooks on command nodes. A command can have persistent pre/post hooks inherited by descendants and local pre/post hooks that apply only to that command.

> [!summary] **Law:** Cross-cutting behavior follows declared ancestry and a declared order. When all ancestor hooks are enabled, pre-hooks move from outer scope to inner scope, the leaf runs, and post-hooks unwind from inner scope to outer scope.

2. The general pattern

For a selected path `r -> a -> ... -> leaf`, the idealized sandwich is:

```
r.PersistentPre
  a.PersistentPre
    ...
      leaf.Pre
      leaf.Run
      leaf.Post
    ...
  a.PersistentPost
r.PersistentPost
```

This is structurally similar to nested middleware or stack unwinding. An outer scope can establish an invariant before an inner scope executes and release or observe it afterward.

3. Cobra's compatibility-sensitive implementation

command.go builds the selected command's parent list inside `execute`. When `EnableTraverseRunHooks` is enabled, persistent pre-hooks are traversed root-to-leaf and persistent post-hooks leaf-to-root. When it is disabled, Cobra preserves legacy behavior: it executes only the first persistent pre-hook found while walking from the leaf upward and only the first persistent post-hook found on the way back.

The package-level switch is declared in cobra.go and defaults to false. This is important architecture evidence: lifecycle semantics are part of compatibility policy, not an implementation detail.

`command_test.go` contains `TestPersistentHooks`, which asserts the full outer-to-inner / inner-to-outer sequence with traversal enabled and the nearest-hook legacy sequence when disabled.

4. Contract and ordering

The reusable pattern requires explicit answers to four questions:

1. **Scope:** Which ancestors contribute middleware to this leaf?
2. **Order:** Does setup run outer-to-inner and cleanup inner-to-outer?
3. **Error semantics:** Which later phases are skipped after an error?
4. **Compatibility:** Can legacy applications opt into a changed traversal rule safely?

Cobra's command post-hooks are not `finally` blocks. If `PreRunE`, flag validation, `RunE`, or `PostRunE` returns an error, later command lifecycle steps may not run. Only package-level finalizers registered through `OnFinalize` are deferred by `execute` after `preRun` completes. A caller must not infer transactional cleanup from the existence of `PostRun`.

5. Negative space

A hook sandwich is not a transaction. It does not roll back leaf side effects. It is not necessarily exception-safe cleanup. It also does not prove that cross-cutting concerns are idempotent when commands are executed more than once in the same process.

The process-global `EnableTraverseRunHooks` switch also means traversal semantics are not configured per command tree. That is a compatibility tradeoff worth noticing when reusing this pattern in libraries intended to host several independent command graphs in one process.

6. When to reuse

Reuse ancestry-scoped hooks where the hierarchy itself carries scope: tenant routes, nested workflows, resource trees, compiler passes, request routers, or policy trees. Prefer explicit middleware objects when hook ordering or failure recovery grows more complex than a small, stable lifecycle.

7. Maturity

Established locally. Hook ordering is implemented and tested. The full traversal behavior is opt-in for compatibility, so consumers should treat the exact semantics as a deliberate mode, not a universal Cobra default.

Source map

- `command.go`
- `cobra.go`
- `command_test.go`
- Pinned repository commit

Related designs

- Root-Owned Dispatch, Leaf-Owned Action
- Injectable Process Boundary for Deterministic Execution

Source file: *designs/04 - Hierarchical Flags with Local Shadowing.md*

Hierarchical Flags with Local Shadowing

Cobra turns the command ancestry into a configuration scope tree. Persistent flags are merged downward, while local flags can intentionally shadow inherited flags.

Pattern statement: Configuration declared persistent on an ancestor flows to descendants, configuration declared local stays local, and a descendant-local name shadows the inherited name when views are assembled.

1. Why this pattern exists

Hierarchical CLIs need both global modifiers and operation-specific modifiers. A root may define `--config` or `--verbose` for every descendant while a leaf defines a local `--format`. The framework needs a rule for visibility and for name collisions that remains understandable in execution, help, completion, and generated docs.

> [!summary] **Law:** Parent-persistent configuration is inherited; child-local configuration is not. A local declaration with the same name is the child's effective declaration and hides the inherited one in the child's local/inherited views.

2. Cobra's concrete mechanism

A `Command` maintains distinct `FlagSet`s for:

- local/effective flags;
- persistent flags declared on that command;
- cached local flags;
- cached inherited flags;
- persistent flags collected from parents.

`mergePersistentFlags` updates the parent persistent set, adds the command's own persistent flags, then adds inherited parent flags to the effective flag set. `InheritedFlags` excludes any parent flag whose name is found in the local flag view. `LocalFlags` explicitly treats a flag as local when it is not the same object as the parent-persistent flag with that name.

Tests in `command_test.go` check that a child sees a parent's persistent flag as inherited while a child-local flag with the same name is classified as local and absent from inherited flags.

3. The general model

Treat each command node as a lexical configuration scope. Let `P(n)` be persistent declarations at node `n`, `L(n)` local declarations, and `Anc(n)` the ordered ancestor path. The effective visible names are approximately:

```
Effective(n) = L(n) union P(n) union inherited(Anc(n))
```

with nearest local declarations winning name conflicts in the child view. The exact runtime value still belongs to the `pflag.Flag` object; the model here is about declaration visibility and ownership.

4. Why shadowing matters

Without shadowing, a global option becomes impossible to specialize. With uncontrolled shadowing, users cannot tell which declaration they are setting. Cobra's help projection exposes local flags and inherited "Global Flags" separately, making scope visible to users.

This is a useful design lesson: if hierarchical configuration is inheritable, its *origin* should remain inspectable. Effective value alone is not enough for debugging.

5. Negative space

Inheritance is not configuration precedence across every source. Cobra's persistent/local flag mechanism does not itself define precedence against environment variables, configuration files, remote settings, or Viper bindings.

Shadowing is also name-based scope, not a type-safe override relationship. Applications should avoid using the same flag name for semantically unrelated concepts merely because the mechanism permits it.

Finally, `TraverseChildren` changes when parent flags are parsed relative to descent; scope visibility and parse timing are related but distinct concerns.

6. When to reuse

This pattern fits nested command trees, policy trees, build target hierarchies, and scoped configuration systems. Reuse it when users benefit from declaring a default once at an outer scope while retaining a clear way for inner scopes to specialize or hide it.

7. Maturity

Established locally. The merge, classification, and shadowing rules are implemented directly in `command.go`, exposed in help/docs, and covered by tests.

Source map

- `command.go`
- `command_test.go`
- Pinned repository commit

Related designs

- Constraint Metadata Drives Both Rejection and Guidance
- Injectable Process Boundary for Deterministic Execution

Source file: *designs/05 - Late-Bound Synthetic Affordances Preserve User Override.md*

Late-Bound Synthetic Affordances Preserve User Override

Cobra lazily creates help/version/completion affordances and checks for user overrides first. The defaults enter the same command/flag model rather than bypassing it.

Pattern statement: Framework-provided affordances should be injected as late as practical, only when absent, and preferably as ordinary model objects so user-defined behavior wins and existing traversal logic can be reused.

1. Why this pattern exists

Frameworks want good defaults, but eager defaults can seize names, mutate models before applications finish composing them, or force users through special escape hatches to replace built-in behavior.

Cobra repeatedly uses the phrase "at the last point possible to allow for user overriding" around help and version initialization. That is an architectural policy.

> [!summary] **Law:** Defaults are guests in the user's semantic model. Insert them late, detect user-owned equivalents first, and express them through the same abstractions as user features.

2. Cobra's synthetic affordances

command.go lazily initializes:

- a `--help` flag only when no help flag exists;
- a `--version` flag only when a version is configured and no version flag exists;
- a `help [command]` subcommand only when the command has subcommands and the application has not provided its own help command.

completions.go lazily initializes:

- the hidden `__complete` command only for completion requests, removing it again when it is not actually the invoked path;
- the user-visible `completion` command unless disabled or already supplied by the application.

These are normal `Command` nodes or normal flags. As a result, standard routing, help grouping, completion, context propagation, and documentation logic can reason about them without a second built-in subsystem.

3. General pattern

A framework default has three phases:

```
declare possibility -> wait for application composition -> synthesize only if still absent
```

This is late binding over a semantic namespace. The framework reserves behavior conceptually but does not preempt the application mechanically.

4. Side-effect containment

The hidden completion command demonstrates a second reason for late synthesis: merely adding a child changes observable structure (`HasSubCommands`, help behavior, routing assumptions). Cobra adds `__complete`, resolves whether that special command is actually being called, and removes it otherwise to avoid changing programs that otherwise consist only of a root command.

This yields a reusable rule: **temporary framework nodes should have a bounded structural lifetime**. If a synthetic node exists only to serve one protocol path, avoid leaving it behind where unrelated projections can observe it.

5. Negative space

Late-bound defaults are not the same as immutable defaults. Cobra mutates the command tree during execution. Code that introspects the graph before initialization may not see the same nodes as code that inspects it during execution.

Also, "user wins" applies only where the framework checks for an override. Other package-level global behaviors remain global configuration rather than synthesized model nodes.

6. When to reuse

Reuse this approach for framework-provided health endpoints, default routes, help panels, system commands, generated admin actions, or fallback handlers. It is particularly effective when the default can be represented by the same public model type users already compose.

Avoid it when runtime mutation of the model would violate concurrency or immutability assumptions. In those systems, perform late synthesis into an immutable execution snapshot instead of mutating the shared graph.

7. Maturity

Candidate ecosystem pattern. Cobra uses the technique consistently across help, version, and completion, and the hidden completion path explicitly manages structural side effects. Cross-framework comparison would be needed for a stronger Garden maturity claim.

Source map

- `command.go`
- `completions.go`
- Pinned repository commit

Related designs

- Command Tree as Semantic Spine
- Completion as an In-Band Protocol

Source file: *designs/06 - Completion as an In-Band Protocol.md*

Completion as an In-Band Protocol

Cobra shell scripts call hidden commands inside the program. The program resolves the real command context and returns completion candidates plus a bitmapped directive over stdout.

Pattern statement: Let the shell delegate semantic completion back to the executable through a stable hidden command protocol; return candidates as data plus an explicit directive so shell adapters do not need to duplicate application semantics.

1. Why this pattern exists

Shell completion is a distributed feature: part of the logic runs in bash/zsh/fish/PowerShell, while the authoritative command semantics live in the application. If generated shell scripts reproduce the command tree and business-specific completion logic, they become stale as soon as the binary changes.

Cobra instead turns completion into a small request/response protocol hosted by the CLI itself.

> [!summary] **Law:** The external shell adapter owns shell mechanics; the application owns semantic completion. Bridge them with a protocol that can express both candidate data and post-processing directives.

2. Protocol shape

completions.go defines hidden request commands:

```
__complete
__completeNoDesc
```

The request carries the partially typed command line as ordinary arguments. Cobra resolves the real command, determines whether completion concerns a subcommand, flag name, flag value, positional value, or custom completion function, and writes one candidate per line.

The final stdout line is:

```
:<directive-integer>
```

`ShellCompDirective` is a bitmap. Bits can tell the shell not to add a space, not to fall back to file completion, to filter file extensions or directories, to preserve order, or that an error occurred. Descriptions are encoded after a tab and can be suppressed by the no-description request mode.

The implementation writes human-oriented diagnostic text to stderr because completion scripts are expected to ignore stderr. That separates machine protocol output from advisory output without inventing another transport.

3. Semantic reuse

The completion engine calls the same `Find`/`Traverse` machinery used for execution, reads the same flags, and consults `ValidArgs` / `ValidArgsFunction`. It also injects help/version flags because completion does not invoke the normal leaf execution path that would otherwise create them.

This is a powerful boundary: shell code stays generic while Go functions can use application state, context, and typed command metadata.

4. Protocol hygiene

Cobra sanitizes completion values before writing them: descriptions can be removed, embedded newlines are cut to the first line, and surrounding whitespace is trimmed. These steps are not cosmetic; a line-oriented protocol must prevent a description from becoming a fake second candidate.

Tests in completions_test.go assert exact protocol output, including candidates, descriptions, hidden/deprecated filtering, and terminal directive values.

5. Negative space

This is not a general secure RPC protocol. Completion runs as the user and may execute application completion callbacks. Expensive or side-effectful completion functions can make the shell slow or surprising.

The protocol is also intentionally line-oriented and compact. It is not appropriate for rich structured responses unless versioning and escaping rules are expanded.

Finally, semantic completion is guidance, not authority. A candidate being suggested does not grant permission, and omitted candidates do not necessarily imply invalidity when the application allows arbitrary args.

6. When to reuse

Reuse this pattern whenever an external adapter needs live semantic advice from an executable but should not embed a second copy of the executable's model: editor completion, launcher menus, command palettes, REPL front ends, or thin GUI shells.

7. Maturity

Established locally. The protocol is central to Cobra's multi-shell completion implementation and is exercised by extensive exact-output tests.

Source map

- `completions.go`
- `completions_test.go`
- `command.go`
- Pinned repository commit

Related designs

- Command Tree as Semantic Spine
- Late-Bound Synthetic Affordances Preserve User Override
- Constraint Metadata Drives Both Rejection and Guidance
- Borrowed Input Slices Stay Read-Only

Source file: *designs/07 - Constraint Metadata Drives Both Rejection and Guidance.md*

Constraint Metadata Drives Both Rejection and Guidance

Cobra stores flag-group relationships as annotations, validates those annotations before running, and also interprets them during completion to require or hide related flags.

Pattern statement: Encode declarative relationships once, then interpret them both as runtime validity checks and as interactive guidance so the user interface does not suggest states that the validator will later reject.

1. Why this pattern exists

A common CLI failure is semantic drift between runtime validation and interactive assistance. Help says two flags are exclusive, completion suggests both, and only after Enter does validation reject the invocation. The problem is not merely bad UX: it reveals two independently maintained models of validity.

Cobra's flag groups encode relationships on the flag objects and reuse those relationships in both runtime validation and completion.

> [!summary] **Law:** A constraint should have one declarative representation. Validators and interactive advisors are separate interpreters of that representation.

2. Cobra's constraint vocabulary

flag_groups.go supports three group relationships:

- **required together:** if any member is set, all members must be set;
- **one required:** at least one member must be set;
- **mutually exclusive:** at most one member may be set.

`MarkFlagsRequiredTogether`, `MarkFlagsOneRequired`, and `MarkFlagsMutuallyExclusive` attach string annotations to each participating `pflag.Flag`. `ValidateFlagGroups` reconstructs group state from those annotations and returns the first deterministic error encountered.

The same file contains `enforceFlagGroupsForCompletion`. Before ordinary completion runs, it interprets the group state as guidance:

- once one flag in a required-together group is present, the remaining flags are marked required so completion offers them;
- if none in a one-required group is present, completion marks all members required so they are offered;
- if one mutually exclusive flag is present, other members are hidden from completion.

Runtime rejection and interactive narrowing therefore come from the same metadata.

3. Evidence

flag_groups_test.go tests success and failure across local flags, persistent flags, inherited flags, subcommands, multiple groups, and deterministic error ordering. completions.go calls `enforceFlagGroupsForCompletion` before generating flag candidates.

4. General pattern

Separate three layers:

```
constraint facts      -> declarative relationships
validation projection -> reject invalid completed states
guidance projection   -> avoid/suggest transitions while state is incomplete
```

The two projections need not behave identically. Validation reasons about a complete invocation; completion reasons about the next possible token. They should, however, share the same constraint facts.

5. Negative space

Constraint metadata is not authorization. A mutually exclusive flag group describes syntax/semantic shape, not whether a user may perform the operation.

This representation is also string-based metadata attached to flags, not a typed constraint graph. Group names are reconstructed from joined flag names. That keeps the API small but limits richer diagnostics and static analysis.

Completion guidance is advisory. A shell user can type anything manually; runtime validation remains the authority.

6. When to reuse

Reuse this pattern in forms, configuration editors, API clients, workflow builders, and any interactive system where invalid combinations can be encoded once and projected into both final validation and incremental guidance.

7. Maturity

Candidate ecosystem pattern. The invariant is strongly implemented and tested inside Cobra. The generalization to other constraint-driven interfaces is compelling but not independently validated in this study.

Source map

- `flag_groups.go`
- `flag_groups_test.go`
- `completions.go`
- `command.go`
- Pinned repository commit

Related designs

- Hierarchical Flags with Local Shadowing
- Completion as an In-Band Protocol
- First-Class Validators and Combinators

Source file: designs/08 - First-Class Validators and Combinators.md

First-Class Validators and Combinators

Cobra defines `PositionalArgs` as a function type and supplies reusable validators plus `MatchAll`, which short-circuits through a sequence of validators.

Pattern statement: Represent argument admissibility as a first-class function and compose small validators to express conjunctions, keeping route selection separate from command-specific argument policy.

1. Why this pattern exists

Routers should find the command. They should not need bespoke branches for every command's arity and positional argument policy. Conversely, leaf handlers should not all reimplement the same "exactly N", "at least N", or "allowed values" checks.

Cobra moves positional admissibility into a small function type:

```
type PositionalArgs func(cmd *Command, args []string) error
```

> [!summary] **Law:** Make validation policy a value that can be attached, reused, tested, and composed independently of dispatch and business effects.

2. Cobra's validator algebra

args.go provides:

- `NoArgs`
- `OnlyValidArgs`
- `NoDuplicateArgs`
- `ArbitraryArgs`
- `MinimumNArgs(n)`
- `MaximumNArgs(n)`
- `ExactArgs(n)`
- `RangeArgs(min,max)`
- `MatchAll(...)`

Some validators are direct functions; parameterized validators are higher-order functions that return a `PositionalArgs`. `MatchAll` forms conjunction by running validators in order and returning the first error.

`Command.ValidateArgs` in command.go dispatches to `ArbitraryArgs` when `Args` is nil or invokes the configured validator otherwise.

3. Small algebra, useful composition

The pattern is deliberately modest. If validators are predicates returning errors, `MatchAll(v1, v2, ...)` models logical conjunction with ordered diagnostics:

```
valid(args) = v1(args) AND v2(args) AND ... AND vn(args)
```

The first failing validator owns the error. This makes validation deterministic and lets applications choose diagnostic priority by ordering validators.

The deprecated `ExactValidArgs` is implemented as `MatchAll(ExactArgs(n), OnlyValidArgs)`, which is a useful migration signal: named special cases can often collapse into composition of orthogonal primitives.

4. Negative space

This is not a full validation language. There is no built-in disjunction combinator, error accumulation, field-path structure, or typed result. Applications with complex semantic validation may need richer domain-specific validators.

`ValidArgs` also serves shell completion, but completion metadata does **not** automatically make values invalid at runtime. Runtime enforcement occurs only when `Args` includes `OnlyValidArgs` or equivalent logic. Suggestion and authority remain distinct.

Validators should ideally be side-effect-free. Cobra's function type does not enforce purity, so applications can violate that discipline; doing so makes completion/testing and retry behavior harder to reason about.

5. When to reuse

This pattern transfers to HTTP request guards, configuration validation, compiler checks, workflow admission, and policy evaluation. It works best when small checks can be named after one invariant and combined near the boundary where inputs become actionable.

6. Maturity

Established locally. The validator type and combinators are public API, directly invoked by the execution pipeline, and used to replace older specialized helpers.

Source map

- `args.go`
- `command.go`
- Pinned repository commit

Related designs

- Constraint Metadata Drives Both Rejection and Guidance

Source file: *designs/09 - Injectable Process Boundary for Deterministic Execution.md*

Injectable Process Boundary for Deterministic Execution

Cobra defaults to `os.Args`, stdin/stdout/stderr, and a background context, but exposes setters and ancestor fallback so the same command graph can execute in tests or embedded hosts.

Pattern statement: Treat process inputs and outputs as overridable execution dependencies - argv, context, stdin, stdout, and stderr - and make child operations inherit those seams unless they explicitly override them.

1. Why this pattern exists

A CLI is naturally coupled to process globals: `os.Args`, `os.Stdin`, `os.Stdout`, `os.Stderr`, and process lifetime. That coupling is convenient for a binary but costly for tests, embedded command runners, notebooks, GUI wrappers, or servers that want to execute commands without replacing global process state.

Cobra puts an overridable seam around those dependencies.

> [!summary] **Law:** Process globals are defaults, not hard dependencies. Execution should accept substitutable argv, context, and I/O channels, and nested operations should inherit the host-provided boundary.

2. Cobra's concrete seam

command.go exposes:

- `SetArgs` for an argv slice;
- `SetContext` / `ExecuteContext` / `ExecuteContextC`;
- `SetIn`;
- `SetOut`;
- `SetErr`.

If no override is set, `ExecuteC` falls back to `os.Args[1:]` and command accessors fall back to the process streams. The internal `getIn`, `getOut`, and `getErr` functions walk to the parent before using the process default, so configuring the root automatically supplies the same I/O boundary to descendants.

Tests in command_test.go build a `bytes.Buffer`, assign it to root output/error, set args directly, and execute the real command graph. Context tests verify that an injected context reaches root, child, and grandchild handlers.

3. Ownership model

Think of execution as a request object split across command fields:

```
Execution environment = argv + context + stdin + stdout + stderr
```

The command graph describes capabilities; the execution environment supplies one run's process-facing dependencies. Cobra stores both on mutable `Command` objects, but the conceptual separation is still valuable.

4. Why ancestor fallback matters

A parent-owned output stream is a scoped dependency, not merely a test convenience. Help generated by a leaf, completion output, error messages, and handler writes through Cobra helpers can all land in the host's selected destination without configuring every child.

This mirrors persistent flag inheritance: hierarchy is used as a scope mechanism for execution dependencies as well as user configuration.

5. Negative space

The `Command` object remains mutable and can retain parsed flag state or other configuration across runs. Injectable I/O does not automatically make one command graph safe for concurrent execution.

The framework also still falls back to global process state when overrides are absent. This is dependency injection by optional override, not constructor-enforced dependency completeness.

Capturing Cobra's output does not capture arbitrary writes performed directly by application code to `os.Stdout` or `os.Stderr`; handlers must use injected writers for the seam to remain complete.

6. When to reuse

Reuse this pattern for testable CLI engines, interpreters, compilers, task runners, and embeddable tools. It is particularly effective when a library has a natural process default but should remain hostable inside another process.

7. Maturity

Candidate ecosystem pattern. Cobra's seams are established and heavily used by its tests; the broader pattern is a well-supported extraction but has not been cross-project validated under this Garden study.

Source map

- `command.go`
- `command_test.go`
- Pinned repository commit

Related designs

- Root-Owned Dispatch, Leaf-Owned Action
- Hierarchical Flags with Local Shadowing
- Borrowed Input Slices Stay Read-Only

Source file: ***designs/10 - Borrowed Input Slices Stay Read-Only.md***

Borrowed Input Slices Stay Read-Only

A Cobra completion bug showed that slicing `os.Args` and later appending a sentinel could overwrite the shared backing array. The fix copies into new storage before speculative append and adds a regression test.

Pattern statement: When a framework receives a caller-owned slice without ownership transfer, treat its backing storage as immutable; copy before any append or mutation whose capacity behavior could write into the caller's array.

1. Why this note exists

This pattern is failure-derived. In April 2026, Cobra fixed a completion bug where internal argument manipulation could mutate `os.Args` supplied by the caller. The failure is small enough to look like a Go gotcha, but the underlying law is architectural: framework inputs have ownership semantics even when Go's type system does not encode them.

> [!summary] **Law:** A borrowed slice is a read-only view unless ownership transfer is explicit. If internal logic may append or mutate, allocate a new backing array first.

2. The failure mechanism

The fix commit is 746ef0715872. Its commit message explains the sequence:

1. completion receives args ultimately derived from `os.Args[1:]` or `SetArgs`;
2. traversal can return sub-slices sharing the same backing array;
3. completion temporarily appends `"--"` as a sentinel;
4. when the sub-slice has spare capacity, Go's `append` writes into the existing backing array;
5. the caller's `os.Args` entry can therefore be replaced by `"--"`.

The current `getCompletions` copies the trimmed arguments explicitly:

```
trimmedArgs := make([]string, len(args)-1)
copy(trimmedArgs, args[:len(args)-1])
```

before later code can append to that working slice.

3. General ownership law

A slice value contains a pointer, length, and capacity. Passing or slicing it copies the header, not the elements. Therefore:

```
borrowed slice header != owned backing array
```

If a callee appends and capacity is sufficient, the append is an in-place mutation from the caller's perspective. If capacity is insufficient, append allocates and the bug disappears. That capacity-dependent behavior is exactly why tests may pass until a different slice shape, traversal path, or compiler/runtime context exposes the alias.

4. Evidence and regression boundary

The fix added `TestCompletionDoesNotMutateOsArgs` in completions_test.go. The test sets `os.Args`, enables `TraverseChildren`, executes a completion request, and asserts that every argument remains unchanged afterward.

This is strong failure evidence: a historical defect, a minimal code change, and a regression test that states the new invariant.

5. Negative space

Not every slice must be copied. Copying defensively at every boundary wastes memory and can obscure legitimate ownership transfer. The relevant questions are:

- Did the caller transfer ownership?
- Will the callee mutate elements?

- Will the callee append, reslice, sort, compact, or reuse capacity?
- Can a derived slice outlive the call or cross goroutines?

A shallow element copy is also insufficient when the elements themselves contain shared mutable pointers, maps, or slices and deep ownership matters.

6. When to reuse

Reuse this rule in parsers, middleware, protocol adapters, batch processors, and any Go API that receives slices as borrowed input. It is especially important in framework code because the caller may reasonably inspect its input after the framework returns.

7. Maturity

Candidate ecosystem pattern. The Cobra-specific invariant is established by a real defect and regression test. The broader ownership rule is fundamental Go engineering, but this Garden study records only this implementation as direct evidence.

Source map

- `completions.go`
- `completions_test.go`
- Pinned repository commit
- Failure-derived commit

Related designs

- Completion as an In-Band Protocol
- Injectable Process Boundary for Deterministic Execution